When Does INT8 Compression Preserve Federated Intrusion Detection on Microcontrollers?When Does INT8 Compression Preserve Federated IDS?

When Does INT8 Compression Preserve Federated Intrusion Detection on Microcontrollers?

Input Ranges, Operating Points, and Client-Data Constraints

Anonymous Author(s)

Abstract

Federated intrusion detectors are trained on server-class hardware but must run on microcontrollers with tens of kilobytes of memory. We ask when compressing such a detector to INT8 preserves its usefulness at a fixed false-alarm rate, why it fails when it does, and which remedies work when only client-held data are available. On CIC-IDS2017 and TON_IoT, with a data pipeline audited for client ownership and test isolation, thresholds chosen on client validation data, and three independent federated runs, 57–67 KB INT8 models detect 98.5% and 99.6% of attacks at a 1% false-alarm budget, at least as many as the float teacher, and a compact model trained federatively from scratch is as good as a pruned and distilled teacher. Failures have one dominant cause, the input quantization range: ten extreme records in a 2,000-record calibration draw drive recall to 0%, and quantization-aware training (QAT) fails for the same reason because it learns that range from unclipped inputs. Bounding the input range removes both failures at no cost. Hidden-layer ranges and the 8-bit output grid matter little; the residual loss comes from coarse activations near the operating point and is recovered by 16-bit activations. The ranking holds on hosts never seen in training. On an ESP32-S3 and an ESP32 the per-tensor INT8 models reproduce the host decisions on all but at most one of 2,700 random and stress inputs per model, at 5.6–6.5 ms per flow (13.2–15.4 ms on an ESP32), with bit-identical outputs on both chips, while models with per-channel weight scales, the converter's default, silently disagree on half of their decisions.

Keywords: federated learning, TinyML, intrusion detection, quantization, microcontrollers

1 Introduction

Intrusion detection on IoT devices is increasingly learned from traffic that clients do not want to pool [1]. Federated learning (FL) trains a shared model from client updates [23, 29], and TinyML runs integer models on microcontrollers [2, 8]. Putting the two together requires a step that is easy to state and easy to get wrong: the federated model has to be compressed, usually to INT8 and a fraction of its size, without pooled data, and the compressed model has to keep detecting attacks at the false-alarm rate an operator can tolerate.

Published federated and TinyML intrusion detectors report accuracy or F1 at a fixed threshold, on splits whose preprocessing often sees the test data, and rarely check the device. We found all three choices in our own earlier pipeline: oversampling before the client partition, thresholds tuned on synthetic records, and a test split that guided method development (§3). After rebuilding the pipeline we ask three questions:

RQ1 When does compression preserve useful detection, under memory budgets of 80 and 128 KB and false-alarm (FAR) budgets of 1% and 5% chosen on client validation data?
RQ2 Why does it fail when it does?
RQ3 Which remedies work under memory and client-data constraints, and what do they cost on the device?

Our answers, on CIC-IDS2017 and TON_IoT with three federated runs per configuration:

- **RQ1.** At 57–67 KB, INT8 models match the float teacher at a 1% FAR budget (98.5% / 99.6% recall). A compact model trained federatively from scratch is a strong baseline that compression must beat; fine-tuning with one client's 10k records suffices (§4).
- **RQ2.** A controlled 2×2 isolates the input range, not hidden ranges, as the cause of collapses; the same mechanism explains why QAT fails; the remaining loss comes from tie groups of coarse INT8 outputs at the operating point (§5).
- **RQ3.** Bounding the input range (clipped or extreme-free calibration, QAT on clipped inputs) removes the failures for free; distillation adds up to 2 points; 16-bit activations recover the tie loss but need a runtime that supports them. On an ESP32-S3, per-tensor INT8 models reproduce the host decisions, while per-channel models silently do not (§6, §7).

We also contribute the audited protocol itself: client-owned partitions before any fitted step, real per-client validation slices, FAR-budget thresholds applied once to a test split disjoint from everything used during development, and an unseen-host split.

2 Background and Related Work

Datasets. CIC-IDS2017 [39] is the most used flow-based IDS benchmark; Engelen et al. [11] document labelling and flow-construction errors in it. TON_IoT [31] and Bot-IoT [24] target IoT traffic.

Federated and lightweight intrusion detection. DÏoT [35] trains per-device-type detectors with FL; Belarbi et al. [4] use one TON_IoT client per IP address; FLAME [36] defends FL against backdoors. Li et al. [25] combine FL and TinyML for intrusion detection on N-BaIoT with feature reduction and quantization. Kitsune [30], HorusEye [10] and NetBeacon [42] run detectors on gateways or switches; we target the endpoint microcontroller. We use FedAvgM [18] in Flower [5].

TinyML and compression. TFLM [8] runs integer models on microcontrollers; MCUNet [26], on-device training [27], SeeDot [14] and MinUn [21] co-design models and runtimes. HeteroFL [9] and FedTiny [19] produce small models during federated training; we compress a trained federated model or train a compact one from scratch. Pruning [16], distillation [17, 37] and quantization [13, 20, 32] are standard; QAT can oscillate [33], and pruned weights re-densify under standard QAT [40]. In FL, quantization mostly compresses updates [38]; Gupta et al. train models robust to later quantization [15].

Calibration ranges. Percentile and KL calibration [41] and the MSE-optimal clip of ACIQ [3] clip activation ranges instead of using extremes; PACT [7] and LSQ [12] learn the clip or step during QAT, and DFQ [34] calibrates without data. These methods target hidden activations; we find that for tabular flow features the *input* range decides the outcome. Deployment faults that yield wrong models without errors are common [6, 22]; §7 adds one for microcontroller runtimes.

3 Experimental Design

3.1 Data, Audit and Splits

We use the eight CIC-IDS2017 CSV files (78 features, binary label) and the TON_IoT network train/test file (37 numeric and label-encoded fields plus six length / digit / symbol counts from the DNS query and URI). Table 1 summarizes the audited pipeline. An audit of our earlier pipeline (Appendix A) found that SMOTE ran on the pooled training set before the client partition (37.5% / 35.1% synthetic rows, with synthetic descendants of one client's records on others), that the validation slice used to choose thresholds was part of a client's training data and 74.6% (CIC-IDS2017 attack rows) / 71.0% (TON_IoT benign rows) synthetic, that 4.7% of synthetic TON_IoT rows carried values impossible for their label-encoded fields, and that the test split had guided method choices. All results in this paper come from the rebuilt pipeline:

1. exact duplicates are removed and feature vectors that occur with both labels are dropped; on CIC-IDS2017, 2,926 records with impossible negative values (overflowed header lengths down to $-3.2 \cdot 10^{10}$, negative durations and inter-arrival times) are dropped as data errors, while the defined −1 sentinel of `Init_Win_bytes` is kept;
2. the test split (20%) is drawn first, only from records that were never in the old test split;
3. records are assigned to four clients (label-balanced) before any fitted step; each client holds out a real, stratified 10% validation slice and undersamples its own training slice to at most 4:1;
4. the scaler uses the mean and variance of the union of client training slices, which a federation can aggregate from per-client sums; validation and test are never used.

Table 1. Audited data pipeline (per dataset). Val. = per-client validation slice (benign count in brackets).

	CIC-IDS2017	TON_IoT
raw records	2,830,743	211,043
data errors dropped	2,926	0
exact duplicates removed	331,190	105,867 (50%)
conflicting-label vectors (records)	719 (7,144)	8 (314)
records kept	2,495,189	105,160
test split (attacks / benign)	84,990 / 414,048	16,212 / 4,820
training slice per client (attacks)	382,455 (76,491)	18,929 (14,592)
val. per client (benign)	49,904 (41,405)	2,103 (482)
features	78	43

For a second test condition (E6) we split TON_IoT by source host: all records of 15 of 51 source IPs form the test split, so no test host is seen in training. CIC-IDS2017 carries no host field in its CSVs.

3.2 Federated Training

All models are MLPs with ReLU, post-ReLU BatchNorm (folded exactly into the next layer before compression, Appendix B) and dropout, trained with Flower and FedAvgM (server momentum 0.5, server learning rate 0.1) for 60 rounds of 3 local epochs, Adam with cosine decay from 10^{-3} to 10^{-4}, class weights and focal loss [28] ($\alpha = 0.35$ on CIC-IDS2017, 0.5 on TON_IoT). The teacher F is 512-256-128. Compact models trained from scratch with the same recipe are S80 (256-128-64, 57 KB / 65 KB as INT8) and S128 (358-179-89, 100 KB / 112 KB). Every configuration is trained with seeds 1, 2 and 3 (E5); a run takes 3–4 min on TON_IoT and 26–41 min on CIC-IDS2017 on one PC.

3.3 Compared Arms

For each teacher seed and memory budget (80 KB: structured pruning keeps 50% of the neurons; 128 KB: 70%) we compare:

F / F8 the float teacher, and the teacher with strong PTQ (189 / 207 KB, over budget; a reference);

P prune → fine-tune 3 FedAvg rounds on the clients' data → strong PTQ;

Table 2. Attack recall (%) at a 1% FAR budget chosen on client validation data, mean ± SD over three federated runs; test FAR in brackets. 80 KB budget; fine-tuning FedAvg over four clients (10k records each). 1-cl.: fine-tuning on one client's 10k records.

arm	KB	TON_IoT	CIC-IDS2017	1-cl. TON_IoT/ CIC-IDS2017
F (FP32)	–	96.68±1.38 [0.84]	96.89±2.05 [0.98]	
F8	189/207	98.37±0.08 [1.18]	99.46±0.14 [0.94]	
P	66/65	96.40±0.92 [0.95]	99.09±0.25 [0.94]	96.92 / 99.06
Q	57/65	55.4±48.8 [0.61]	81.9±9.2 [0.80]	59.6 / 88.5
QC	57/65	97.86±0.44 [1.01]	99.57±0.10 [0.94]	
R	57/65	**98.51**±0.11 [1.16]	**99.60**±0.15 [0.92]	97.80 / 99.52
S80	57/65	97.73±0.64 [0.98]	99.54±0.22 [0.92]	
SQ	57/65	59.0±51.1 [0.55]	84.8±5.4 [0.82]	
SQC	57/65	97.71±1.09 [1.01]	99.59±0.07 [0.97]	
128 KB budget: P 97.78 / 99.19, QC 98.24 / 99.56, R 98.72 / 99.64, S128 98.58 / 99.37				

- **Q** the same pruned and fine-tuned model → QAT for 2 rounds → INT8;
- **QC** as Q, but QAT on inputs clipped to ±5 standard deviations;
- **R** prune → fine-tune distilled from F (temperature 2, $\alpha = 0.5$) on clipped inputs → QAT on clipped inputs → INT8;
- **S / SQ / SQC** the from-scratch compact model with strong PTQ / QAT / QAT on clipped inputs.

Strong PTQ builds seven calibrations from 2,000 records of client 0 (TFLite min/max with and without ±5 SD input clipping, per-tensor 99.9 / 99.99th percentile, MSE [3], KL [41], clipped 99.99th percentile) and keeps the one with the best mean validation recall at the two FAR budgets. Fine-tuning uses 10k records per client, either FedAvg over all four clients or one client alone (the client-data constraint).

3.4 Operating Points and Metrics

For every model and stage, the threshold is the smallest one whose FAR on the pooled client validation slices is at most the budget (1% or 5%). It is frozen and applied once to the test split; we report attack recall and test FAR, mean ± SD over the three teacher seeds. Decision transitions (E2) compare each test attack's decision at consecutive stages (teacher, compact FP32, QAT float, INT8), each at its own frozen threshold. Thresholds are computed in float32: with NumPy 1.x a float64 threshold is cast back to float32 in the comparison, which silently re-admits a whole tie group of INT8 outputs and let validation FAR reach 44% at a 5% budget in our first run.

4 RQ1: When Compression Preserves Detection

Table 2 gives attack recall at a validation-chosen 1% FAR. At 5% every arm except the QAT arms Q and SQ lies within 0.2 points of the teacher (99.7–99.9% on TON_IoT, 99.8–99.98% on CIC-IDS2017).

Table 3. TON_IoT, unseen source hosts (E6): recall (%) at a 1% validation FAR budget, 80 KB, three runs.

F	F8	R	S80	QC
98.20±0.11	98.08±0.18	96.57±0.62	96.43±0.92	96.30±0.87
SQC	P	Q	SQ	
95.96±1.60	94.79±1.42	52.9±45.8	56.9±49.2	

Compression preserves detection when the input range is bounded. Every arm whose input range is bounded (P with strong PTQ, QC, R, S, SQC) matches the teacher's recall at 1% FAR within one SD, and R, QC, S and SQC exceed it, on both datasets and at both budgets. The 128 KB budget adds at most 1.4 points over 80 KB. The two QAT arms that learn their input range from unclipped data (Q, SQ) fail; §5 explains why.

The float teacher's 1% operating point is the least stable. F has the largest SD (1.4 and 2.1 points). On CIC-IDS2017 about 2,265 PortScan flows per run sit just below its 1% threshold, and every compact or INT8 model recovers them (§5.1). This is an operating-point sensitivity of the teacher, not a benefit of compression (§5.3).

A compact model trained from scratch is as good as compression. S80 matches P and R on CIC-IDS2017 and lies between them on TON_IoT; at 128 KB S128 matches R within 0.3 points. Under a memory budget known at training time, a second federated training of a compact model is a strong baseline.

The client-data constraint is not binding. Fine-tuning P and R on one client's 10k records changes recall by at most 0.8 points relative to fine-tuning across all clients (Table 2); only the failing Q arm moves more, in both directions.

Unseen hosts (E6). On the host-grouped TON_IoT split (Table 3) compression costs 1.6–1.9 (R, S, QC) to 3.4 points (P) relative to the teacher, more than on the random split, with the same ranking; the QAT collapse and its repair reproduce. The FAR budget chosen on validation does not transfer to unseen hosts for the float models (test FAR 3.2% at a 1% budget), while the INT8 models stay below it; with 2,537 benign test records from few hosts this FAR estimate is noisy.

5 RQ2: Why Compression Fails

5.1 Where Decisions Change (E2)

Table 4 counts the test attacks whose decision changes between stages. Pruning and fine-tuning move thousands of decisions in both directions; the INT8 conversion moves few, except when the input range is wrong. For Q the loss happens in QAT itself, before conversion: 45,603 attacks (three runs) are lost between the compact FP32 and the QAT float

Table 4. Test attacks lost / recovered between stages at the 1% FAR operating point (sum over three runs, 80 KB). Families with the most losses in brackets.

arm	stage	lost	rec.	main losses
CIC-IDS2017				
F8	teacher→INT8	573	7,125	DoS Hulk 384, Bot 73
P	teacher→pruned+FT	759	7,124	DoS Hulk 213, Web brute force 185
P	FP32→INT8	1,040	268	Web brute force 360, DoS Hulk 250
R	FP32→INT8	127	532	DoS Hulk 56, GoldenEye 32
Q	FP32→QAT float	45,603	21	PortScan 17,713, DDoS 13,079
Q	QAT float→INT8	15	1,051	
TON_IoT				
P	teacher→pruned+FT	859	645	dos 321, xss 197, mitm 103
P	FP32→INT8	336	415	dos 225, ddos 46
R	FP32→INT8	104	46	scanning 73

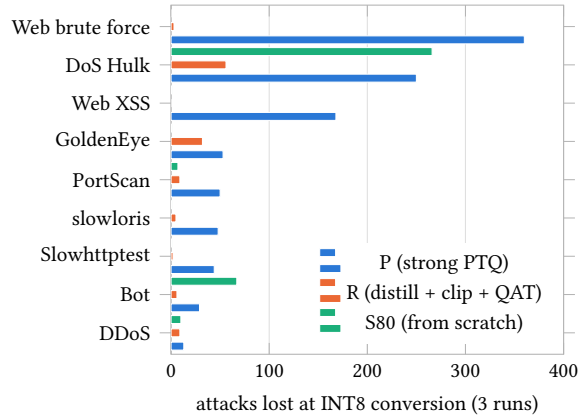


Figure 1. E2 on CIC-IDS2017: test attacks that the compact FP32 model detects and its INT8 version misses, at the 1% FAR operating point, by family (80 KB, sum over three runs). Totals: P 1,040, R 127, S80 369.

model and 15 in the conversion. R's conversion loses 127 attacks where P's loses 1,040 (Figure 1): P's losses concentrate in the small web-attack families (brute force 360, XSS 168) and DoS Hulk, which R keeps; S80 loses mostly DoS Hulk and Bot flows. Two of P's three largest losses are among the smallest families (1,507 web brute-force and 652 XSS records in the whole dataset): aggregate recall hides where a coarse INT8 model gives up.

5.2 Input Range versus Hidden Range (E3)

To isolate the cause we fix the compact FP32 model and the calibration draw and vary only the ranges, exporting per-tensor fake-quantized models: input range min/max or clipped to ±5 SD, crossed with hidden ranges min/max or 99.99th percentile. Each cell is calibrated with three random 2,000-record draws, the same draws with the client's ten most extreme records swapped in, and draws without any record beyond 10 SD (Figure 2, Table 5).

Table 5. E3: recall (%) at 1% FAR of the pruned and fine-tuned 80 KB model, by input range (rows A/C: min/max; B/D: clipped ±5 SD) and hidden range (A/B: min/max; C/D: 99.99th percentile), mean over three runs and three draws. Step: input quantization step of A, in SD.

	calibration draw	A	B	C	D	step
CIC-IDS2017	random	70.9	98.9	73.9	99.1	0.15
	+ 10 extreme records	**0.0**	98.9	12.7	99.0	4.68
	none beyond 10 SD	98.0	99.0	98.5	99.0	0.05
TON_IoT	random	63.5	96.3	63.6	96.3	0.53
	+ 10 extreme records	23.3	96.1	23.3	96.2	0.74
	none beyond 10 SD	96.4	96.6	96.4	96.6	0.05

The input range decides the outcome; the hidden range barely matters (A≈C, B≈D), and fewer than 0.05% of hidden values saturate in any cell. Ten extreme records among 2,000 widen the CIC-IDS2017 input step to 4.7 SD, so ordinary flows quantize to a handful of levels and every attack is missed. Clipping the input range makes the result independent of the draw. The teacher and the from-scratch models behave the same. QAT fails for the same reason: its input quantizer tracks the moving min/max of unclipped training batches, so it learns the wide range itself, and QAT on clipped inputs (QC, SQC) is as good as the best PTQ arms (Table 2). Clipping has a cost: ±5 SD saturates at least one feature in 28% of CIC-IDS2017 test rows (7.6% on TON_IoT), and for the CIC-IDS2017 teacher B and D are 1.5 points below extreme-free min/max calibration (A, last row), the gentler remedy.

5.3 Output and Activation Resolution (E4)

Table 6 separates the remaining INT8 effects. Rounding only the FP32 output to the INT8 output grid (step 1/256) costs at most 0.25 points: the output grid is not the problem. The INT8 models, however, produce few distinct outputs on the validation slices (23–35 for the pruned CIC-IDS2017 models vs. 241–245 for the rounded FP32 output), so up to about 1,000 benign validation records share the output value just below the threshold. The 1% budget can then only be met by jumping over the whole tie group (validation FAR 0.74% instead of 1%), which costs recall. Exporting the logit instead of the sigmoid gives identical decisions: the bottleneck is the INT8 logit and the activations before it. With 16-bit activations and INT8 weights the tie group shrinks to about 10 records and most of the loss returns (CIC-IDS2017: 462 lost attacks become 131). On TON_IoT, where INT8 gains over FP32, clipping the FP32 inputs to ±5 SD gives the same gain: it is input saturation. On CIC-IDS2017 the INT8 gain of F and S80 (+2.1–2.5 points, the PortScan cluster of §4) disappears with 16-bit activations and is not reproduced by clipped FP32 inputs: INT8 rounding moves those flows across the teacher's threshold, an operating-point effect rather than a robust improvement.

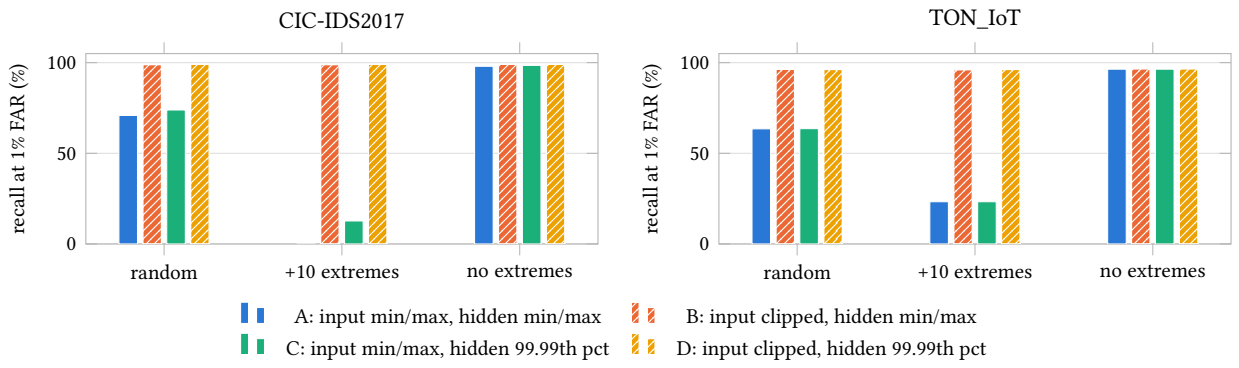


Figure 2. E3: recall of the pruned and fine-tuned 80 KB model when only the quantization ranges change (same FP32 model and calibration draw; mean over three runs and three draws). Hatched bars clip the input range to ±5 SD. Values in Table 5.

Table 6. E4: recall (%) at 1% FAR, three runs. Clip: FP32 with inputs clipped to ±5 SD. Grid: FP32 with its output rounded to the INT8 grid. 16-bit: 16-bit activations, INT8 weights.

	model	FP32	clip	grid	INT8	16-bit
TON_IoT	teacher F	96.68	98.40	96.65	98.28	98.40
	pruned+FT	96.24	97.35	96.01	96.59	97.34
	S80	96.98	97.52	96.90	97.36	97.52
CIC-IDS2017	teacher F	96.89	96.76	96.76	99.40	96.21
	pruned+FT	99.39	99.18	99.28	98.99	99.29
	S80	97.31	97.18	97.26	99.45	96.73

6 RQ3: Remedies and Their Cost

Table 7 lists what worked, its effect and its cost. Bounding the input range is the remedy that matters: it is free on the device, needs no extra client data, and turns the two failure modes (min/max calibration with extreme records, QAT on unclipped inputs) into the best arms. Excluding extreme records from the calibration draw is the gentler variant for models that use the tails. Distillation helps on TON_IoT at 80 KB (+2.1 over P) and adds 0.5 on CIC-IDS2017. Training a compact model from scratch removes the compression step altogether at the price of a second federated training (26 min vs. 40 min for the CIC-IDS2017 teacher). 16-bit activations remove the remaining tie loss but enlarge the arena and, as §7 shows, need runtime support. Fine-tuning data were never the limiting resource: one client's 10k records suffice.

7 On-Device Agreement and Cost (E7)

A host result is only useful if the device reproduces it. We stream test inputs over serial to firmware (PlatformIO, TensorFlowLite_ESP32 1.0.0, TFLM reference kernels) that runs each embedded model and returns the exact float32 output bits and per-stage timings, and compare with the host TFLite interpreter. Per model, the inputs are 1,500 random test records, the test records within two output steps of a validation threshold of any model, 800 records with a feature beyond 5 SD, 400 records with one feature set to ±50 or ±1,000 SD, and 10 zero vectors (4,462 on TON_IoT, 5,902 on CIC-IDS2017).

Table 7. Remedies, effect on recall at 1% FAR (80 KB) and cost on an ESP32-S3.

remedy	effect	cost
clip calibration inputs ±5 SD, or drop records beyond 10 SD	CIC-IDS2017 0.0→98.9–99.0 with extreme records in the draw	none on device
QAT on clipped inputs (QC, SQC)	TON_IoT 55.4→97.9, CIC-IDS2017 81.9→99.6	none
distillation from F (R vs. P)	TON_IoT +2.1, CIC-IDS2017 +0.5	teacher pass over client data
compact model from scratch (S)	equal to P/R on CIC-IDS2017	second federated training
16-bit activations	CIC-IDS2017 lost attacks 462→131	arena 1.4→1.8 KB, +14% latency, runtime support
per-tensor weight scales	device agreement (§7)	none

Table 8 shows two classes of models. Per-tensor INT8 models reproduce the host: at least 96% of outputs are within one output step, all random, out-of-range and synthetic-extreme inputs get the same decision (one exception in 2,700 per model), and the few disagreements are inputs we chose to lie within two steps of the threshold. Models whose Dense weights carry per-channel scales, the TFLite converter's default and what strong PTQ selected for two TON_IoT models, run without any error on this runtime but disagree with the host on half of the random records: the runtime ignores the per-channel scales. Re-exporting the same models with per-tensor scales restores agreement (4,462 / 4,462). The 16-bit-activation remedy of §5.3 is likewise not reproduced and

Table 8. E7: share of outputs within one INT8 output step of the host, decisions agreeing at the 1% FAR threshold on all inputs and on the 1,500 random records, and median invoke latency on an ESP32-S3 and an ESP32 (both 240 MHz). The two chips return bit-identical outputs. Arena 1.4 KB (1.8 KB for 16-bit).

model	weights	≤1 step	all inputs	random	S3 ms	ESP32 ms
TON_IoT R80	per-tensor	97.9%	4,462/4,462	1,500/1,500	5.56	13.19
TON_IoT QC80	per-tensor	97.4%	4,456/4,462	1,500/1,500	5.55	13.20
TON_IoT S80, per-tensor export	per-tensor	99.2%	4,462/4,462	1,500/1,500	5.56	13.35
TON_IoT P80, per-tensor export	per-tensor	99.8%	4,461/4,462	1,500/1,500	5.55	13.18
TON_IoT S80 (strong PTQ)	per-channel	4.9%	2,209/4,462	443/1,500	5.56	13.21
TON_IoT P80 (strong PTQ)	per-channel	40.9%	3,141/4,462	705/1,500	5.56	13.29
CIC-IDS2017 R80	per-tensor	96.4%	5,882/5,902	1,500/1,500	6.48	15.38
CIC-IDS2017 S80	per-tensor	97.2%	5,898/5,902	1,500/1,500	6.47	15.37
CIC-IDS2017 P80	per-tensor	97.9%	5,878/5,902	1,499/1,500	6.47	15.36
CIC-IDS2017 16-bit act.	per-channel	67.5%	4,952/5,902	1,425/1,500	7.39	16.11

needs a newer TFLM build. Host-side evaluation of such models says nothing about the device, which is why we recommend this streaming check before reporting on-device accuracy. The ESP32 (Xtensa LX6) returns bit-identical outputs to the ESP32-S3 (LX7) for all 14 models and 72,548 inputs, so every deviation from the host comes from the runtime, not the chip; it ran the whole stream without a reset, while the S3 restarted a few times at random inputs and resumed. A flow takes 5.6 ms (TON_IoT, 43 features) and 6.5 ms (CIC-IDS2017, 78 features) on the ESP32-S3 and 13.2 ms and 15.4 ms on the ESP32; input quantization takes 3–13 μs and the arena 1.4 KB.

8 Recommendations

1. Choose thresholds at a FAR budget on real, client-held validation records, never on oversampled ones, and compute them in the output's own precision (§3.4).
2. Bound the input range before PTQ and QAT: clip the calibration and QAT inputs or drop extreme records from the calibration draw. Check the exported input step in standard deviations: in our models a step of 0.15 SD or more always collapsed recall at 1% FAR, and 0.05 SD never did.
3. Report a compact model trained federatively from scratch at the same budget; it is the baseline a compression pipeline has to beat.
4. Report losses per attack family: small families are lost first and aggregate recall hides it.
5. Export per-tensor weights unless the device runtime is known to support per-channel scales, and verify on the device by streaming inputs near the threshold and beyond the input range.

9 Threats to Validity

Scope: two datasets, binary detection, one model family (MLP) and four clients. *Splits*: CIC-IDS2017 has no host field, so the unseen-host test is TON_IoT-only, and its benign test set comes from few hosts, making test FAR noisy. *Seeds*: three federated runs per configuration; the float teacher's 1%-FAR recall varies by up to 2 points across runs, and claims about it are stated as sensitivity. *Labels*: CIC-IDS2017 contains labelling errors [11]; we drop impossible values and conflicting labels but do not relabel. *Runtime*: one TFLM release with reference kernels on two ESP32 chips; optimized kernels change latency and newer releases support per-channel and 16-bit models. *Threat model*: no adversarial or poisoning study [36].

10 Conclusion

A federated intrusion detector can be compressed to 57–67 KB INT8 and keep its detection at a 1% false-alarm budget on two datasets, three runs and unseen hosts, using only client-held data, and a compact model trained federatively from scratch does as well. When compression fails, the input quantization range is the cause, for post-training quantization and QAT alike, and bounding it is free. The remaining loss sits in coarse activations at the operating point. Finally, a model that is correct on the host can be silently wrong on the microcontroller: per-tensor INT8 models agree with the host on the device, per-channel ones do not on the runtime we used.

References

[1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The Internet of Things: A Survey. *Computer Networks* 54, 15 (2010), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

[2] Colby R. Banbury, Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, Robert Hurtado, David Kanter, Anton Lokhmotov, David Patterson, Danilo Pau, Jae sun Seo, Jeff Sieracki, Urmish Thakker, Marian Verhelst, and Poonam Yadav. 2020. Benchmarking TinyML Systems: Challenges and Direction. *arXiv preprint arXiv:2003.04821* (2020).

[3] Ron Banner, Yury Nahshan, and Daniel Soudry. 2019. Post Training 4-bit Quantization of Convolutional Networks for Rapid-Deployment. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] Othmane Belarbi, Theodoros Spyridopoulos, Eirini Anthi, Ioannis Mavromatis, Pietro Carnelli, and Aftab Khan. 2023. Federated Deep Learning for Intrusion Detection in IoT Networks. In *IEEE Global Communications Conference (GLOBECOM)*. 237–242. https://doi.org/10.1109/GLOBECOM54140.2023.10437860

[5] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. 2020. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2020).

[6] Zhenpeng Chen, Huihan Yao, Yiling Lou, Yanbin Cao, Yuanqiang Liu, Haoyu Wang, and Xuanzhe Liu. 2021. An Empirical Study on Deployment Faults of Deep Learning Based Mobile Applications. In *IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*.

[7] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. PACT: Parameterized Clipping Activation for Quantized Neural Networks. *arXiv preprint arXiv:1805.06085* (2018).

[8] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Tiezhen Wang, Pete Warden, and Rocky Rhodes. 2021. TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems. In *Proceedings of Machine*

Learning and Systems (MLSys), Vol. 3. 800–811.

[9] Enmao Diao, Jie Ding, and Vahid Tarokh. 2021. HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients. In *International Conference on Learning Representations (ICLR)*.

[10] Yutao Dong, Qing Li, Kaidong Wu, et al. 2023. HorusEye: A Real-time IoT Malicious Traffic Detection Framework using Programmable Switches. In *32nd USENIX Security Symposium*.

[11] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. In *IEEE Security and Privacy Workshops (SPW)*. 7–12. https://doi.org/10.1109/SPW53761.2021.00009

[12] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. 2020. Learned Step Size Quantization. In *International Conference on Learning Representations (ICLR)*.

[13] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv preprint arXiv:2103.13630* (2021).

[14] Sridhar Gopinath, Nikhil Ghanathe, Vivek Seshadri, and Rahul Sharma. 2019. Compiling KB-Sized Machine Learning Models to Tiny IoT Devices. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

[15] Kartik Gupta, Marios Fournarakis, Matthias Reisser, Christos Louizos, and Markus Nagel. 2023. Quantization Robust Federated Learning for Efficient Inference on Heterogeneous Devices. *Transactions on Machine Learning Research (TMLR)* (2023). arXiv:2206.10844.

[16] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations (ICLR)*.

[17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).

[18] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. 2019. Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv preprint arXiv:1909.06335* (2019).

[19] Hong Huang, Lan Zhang, Chaoyue Sun, Ruogu Fang, Xiaoyong Yuan, and Dapeng Wu. 2023. Distributed Pruning Towards Tiny Neural Networks in Federated Learning. In *IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. 190–201.

[20] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.

[21] Shikhar Jaiswal, Rahul Kranti Kiran Goli, Aayan Kumar, Vivek Seshadri, and Rahul Sharma. 2023. MinUn: Accurate ML Inference on Microcontrollers. In *Proceedings of the 24th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*. 26–39.

[22] Purvish Jajal, Wenxin Jiang, Arav Tewari, Erik Kocinare, Joseph Woo, Anusha Sarraf, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2024. Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. https://doi.org/10.1145/3650212.3680374

[23] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. 2016. Federated Learning: Strategies for Improving Communication Efficiency. *arXiv preprint arXiv:1610.05492* (2016).

[24] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. 2019. Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset. *Future Generation Computer Systems* 100 (2019), 779–796. https://doi.org/10.1016/j.future.2019.05.041

[25] Mingyan Li et al. 2025. Cognitive IoT and Edge Computing for Intrusion Detection with Federated TinyML. In *IEEE World AI IoT Congress (AIIoT)*. 677–684.

[26] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: Tiny Deep Learning on IoT Devices. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[27] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. 2022. On-Device Training Under 256KB Memory. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[28] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *IEEE International Conference on Computer Vision (ICCV)*. 2980–2988.

[29] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 54)*. 1273–1282.

[30] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Network and Distributed System Security Symposium (NDSS)*.

[31] Nour Moustafa. 2021. A New Distributed Architecture for Evaluating AI-Based Security Systems at the Edge: Network TON_IoT Datasets. *Sustainable Cities and Society* 72 (2021), 102994. https://doi.org/10.1016/j.scs.2021.102994

[32] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. 2021. A White Paper on Neural Network Quantization. *arXiv preprint arXiv:2106.08295* (2021).

[33] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. 2022. Overcoming Oscillations in Quantization-Aware Training. In *Proceedings of the 39th International Conference on Machine Learning (ICML) (PMLR, Vol. 162)*. 16318–16330.

[34] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. 2019. Data-Free Quantization Through Weight Equalization and Bias Correction. In *IEEE/CVF International Conference on Computer Vision (ICCV)*. 1325–1334.

[35] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, and Ahmad-Reza Sadeghi. 2019. DÏoT: A Federated Self-learning Anomaly Detection System for IoT. In *IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. 756–767.

[36] Thien Duc Nguyen, Phillip Rieger, Huili Chen, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, Azalia Mirhoseini, Shaza Zeitouni, Farinaz Koushanfar, Ahmad-Reza Sadeghi, and Thomas Schneider. 2022. FLAME: Taming Backdoors in Federated Learning. In *31st USENIX Security Symposium*.

[37] Antonio Polino, Razvan Pascanu, and Dan Alistarh. 2018. Model Compression via Distillation and Quantization. In *International Conference on Learning Representations (ICLR)*.

[38] Amirhossein Reisizadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. 2020. FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 108)*. 2021–2031.

[39] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. https://doi.org/10.5220/0006639801080116

[40] TensorFlow Model Optimization. 2021. Collaborative Optimization: Pruning-Preserving Quantization Aware Training.

https://www.tensorflow.org/model_optimization/guide/combine/collaborative_optimization.

[41] Hao Wu, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. *arXiv preprint arXiv:2004.09602* (2020).

[42] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. 2023. An Efficient Design of Intelligent Network Data Plane. In *32nd USENIX Security Symposium*.

A Audit of the Earlier Pipeline

Checks (docs/AUDIT.md): **D1** record ids and split indices recoverable (fixed: per-run manifests with ids and hashes); **D2** threshold-validation records excluded from training (failed: the slice was part of client 0's training data); **D3** validation uses real records (failed: 37.8% / 35.1% synthetic); **D4** no record or synthetic descendant crosses a partition (failed: SMOTE before partitioning, ~255k / 11.4k synthetic rows per client); **D5** scaler fit on training data only (passed); **D6** test split untouched by development (failed; a fresh random split still shared 20% of its records with the old one, so the new test split excludes them); **D7** duplicates and conflicting labels (fixed); **C3** SMOTE on label-encoded fields (failed: 4.7% of synthetic TON_IoT rows impossible); **N1** extreme records (the most extreme CIC-IDS2017 records were overflow errors, now dropped); **N2** input conversion (passed); **N3** BatchNorm folding (passed); **N4** no test data in training (passed); **N5** seeds recorded (fixed). Results of the earlier pipeline were exploratory and are not reported here.

B Exact BatchNorm Folding

Each hidden layer is Dense→ReLU→BatchNorm→Dropout. BatchNorm at inference is $s \odot h + t$ with $s = \gamma/\sqrt{\sigma^2 + \varepsilon}$ and $t = \beta - s \odot \mu$. Since it follows the ReLU it is folded into the next Dense layer (W, b): $W' = \mathrm{diag}(s)W$, $b' = b + t^\top W$ (deviation $\leq 2 \cdot 10^{-5}$). Folding it into the preceding layer, a common shortcut, changed 11.6% of decisions on our layout.

C Toolchain Pitfalls

Six failures produced plausible numbers without any error: int8 weight exchange without scales in FL; QAT-wrapper stripping that copied no weights; BatchNorm folded on the wrong side; pruning that reset BatchNorm statistics; float64 thresholds compared with float32 INT8 outputs (§3.4); and per-channel weight scales ignored by the microcontroller runtime (§7). The checks that catch them: dequantize-requantize equality at the receiver, copy-by-name with output comparison, equivalence assertions after folding, validation FAR asserted against the budget, and device streaming against the host.